An Unsupervised Malicious Web Request Detection Based on Transformer and Contrastive Learning

Shiming He, Ying Zhang, Diqing Liang, and Pradip Kumar Sharma, *Senior Member, IEEE*

***Abstract*—The World Wide Web (Web) is a crucial part of the Internet. Web attacks are becoming more and more serious and complex. Malicious Web request detection aims to rapidly and accurately identify abnormal attacks on the network. Deep learning is being applied to malicious Web request detection, resulting in high detection performance. However, most deep learning-based methods are supervised and ignore special characters, which are hard to detect unknown malicious Web requests. The labels of Web request are fewer and Web request data is insufficient. Therefore, we propose an unsupervised malicious Web request detection based on transformer and contrastive learning (UTCDetector). UTCDetector exploits preprocessing and 2-gram word segmentationto preserve special characters, extracts semantic feature by Transformer, and leverages hypersphere loss function and contrastive learning to handle insufficient Web data without abnormal label. Since the public Web request datasets (CSIC 2010, CSIC TORPEDA 2012, and ECML/PKDD 2007) were created before 2012, we collected Web requests from a university Web application server in 2023 to build a private dataset named School 2023. This dataset contains more modern and complex attacks. The experimental results on the four datasets demonstrate that our method achieves a higher F1-score than other existing methods and ablation variants.**

***Index Terms*—Malicious Web request, unsupervised, transformer, contrastive learning, special characters.**

I. Introduction

THE WORLD Wide Web (Web) is a crucial part of the Internet and encompasses various Web applications that have changed our daily lives, including search engines, social networks, and online payment systems. Web attacks are becoming more and more serious and complex, often endangering the security of cyberspace and causing great security risks to society. According to the "2022 Interconnection Security Report" [1] issued by the China National Internet Emergency Center, Web attack incidents in China continued to maintain a high level in 2022, with an increase of nearly 30% compared to 2021. Web attack includes Web page tampering, drive-by download attack, malicious domain names, webpage impersonation, and website backdoors. These attacks expose user information such as usernames and passwords, disrupt the normal website service, cause economic losses, and negatively impact user experience. Malicious Web request detection aims at quickly and accurately identifying abnormal attacks in the network [2]. Therefore, in the face of increasingly complex and diverse Web attacks, an efficient and accurate malicious Web request detection system is very important.

```
1: GET http://www.xyz.com/ index.php?uid=>< body onload= alert('test1')>
&pwd=test  HTTP/1.1
2: User-Agent: Mozilla/5.0
3: Pragma: no-cache
……
```

Fig. 1. A malicious Web request.

Web request mainly includes Uniform Resource Location (URL), request header, request body, and status information. When a website is attacked, the attack behaviors may be represented in Web request, such as request with unusual length, abnormal session cookies, abnormal response times, or abnormal response states. To bypass firewalls and Web application protection systems, malicious actors often modify malicious requests by alternative representations that serve the same purpose, which are termed Complex Malicious Web Request (CMWR). For example, when attack requests undergo coding and encryption, they frequently manage to bypass detection mechanisms. The reason is that code and encryption introduce special characters to replace the original ones. The attack information is hidden in these special characters. Fig. 1 illustrates a malicious Web request, which contains an additional ">" symbol in the request section of the URL. It belongs to the Cross-Site Scripting (XSS) attack. XSS attack involves the injection of malicious JavaScript code that can exfiltrate user data, compromise user accounts, and even lead to website backdoors in certain cases.

The traditional methods of malicious Web request detection can be divided into two categories: rule-based approaches [3] and behavior-based approaches [4]. The rule-based approaches

Received 15 August 2024; revised 24 March 2025; accepted 11 April 2025. Date of publication 21 April 2025; date of current version 7 August 2025. This work is supported in part by the National Natural Science Foundation of China under Grants 62272062, the Science and Technology Innovation Program of Hunan Province under Grant 2023RC3139, the Natural Science Foundation of Hunan Province 2025JJ50373, the Scientific Research Fund of Hunan Provincial Transportation Department under Grant 202143. The associate editor coordinating the review of this article and approving it for publication was Y. Miao. *(Corresponding author: Ying Zhang.)*

Shiming He and Diqing Liang are with the School of Computer Science and Technology and the Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation, Changsha University of Science and Technology, Changsha 410114, China (e-mail: smhe_cs@csust.edu.cn).

Ying Zhang is with the School of Computer Science and Technology and the Hunan Provincial Key Laboratory of Intelligent Processing of Big Data on Transportation, Changsha University of Science and Technology, Changsha 410114, China, and also with the Institute of Information Engineering, Hunan University of Science and Engineering, Yongzhou 425100, China (e-mail: zhangying@stu.csust.edu.cn).

Pradip Kumar Sharma is with the Department of Computing Science, University of Aberdeen, AB24 3UE Aberdeen, U.K. (e-mail: Pradip.sharma@abdn.ac.uk).

Digital Object Identifier 10.1109/TNSM.2025.3563089

establish a model based on known attack patterns and identify potential attacks by matching observed behaviors with corresponding attack features. The behavior-based approaches, in contrast, develop a profile of acceptable behavior and treat any deviation as an attack. In practice, updating rules for new attacks is a labor-intensive and time-consuming task. Notably, these methods require sufficient features and representations of abnormal or normal behaviors. Therefore, the detection effectiveness of malicious Web requests is not ideal.

Deep learning and machine learning techniques have revolutionized the field of data analysis and time series anomaly detection [5], [6], [7], [8], by enabling the automatic extraction of intricate patterns, thereby facilitating the accurate and efficient differentiation of diverse object types. Therefore, various machine learning methods have been used to detect malicious Web requests, such as HTTP query string to numeric (HQTN) [9], Support vector machines (SVM) optimization [10], and open-source one-class SVM [11]. These methods manually select features, such as attribute name, attribute value, attribute length, attribute character distribution, structural inference, attribute order, and so on. Manual feature selection relies on expert knowledge and consumes significant time and resources.

To automatically extract features, deep learning is applied in malicious Web request detection. Since the Web request contains a lot of words, many methods use Natural Language Processing (NLP) techniques such as N-Gram, Word Embeddings [12], [13] to extract feature, and exploit Long Short Term Memory (LSTM), Auto-Encoder, and Recurrent Neural Network (RNN) [14] to detect abnormal behaviors, such as ELSV [15], LogBERT-BiLSTM [16], DeepWAF [17], Autoencoder [18], and MF [19]. Although these methods achieve high performance, malicious Web request detection still faces the following problems.

- **It is hard to detect unknown malicious Web requests.** Although most methods [15], [16], [17] based on deep learning consider the semantic feature, they are supervised and require abnormal label, which only identify known attacks. The unsupervised approach based on autoencoders [18] overlooks semantic features by treating URLs as character sequences based on ASCII codes. It achieves lower performance compared with the supervised methods. Generally, malicious requests have a longer value length [9] due to the path, repetitive patterns or periodic structures. It is necessary to tackle long sequence data and mine the relationships in long sequence data, named by long-term dependencies. However, most existing methods exploit Convolutional Neural Network (CNN) and LSTM. CNN model is hard to handle complex semantic relationships. LSTM model is time-inefficient to extract the long-term dependencies due to its sequential processing nature.
- **The labels of Web requests are fewer.** Generally, normal and authorized access behavior generate a significant volume of regular Web requests. The Web requests from both attackers and legitimate users exhibit similarities, making it arduous for people lacking extensive knowledge and experience to differentiate between them accurately. Manually labeling Web requests is both time-consuming and labor-intensive. Additionally, supervised methods are incapable of identifying new, unknown malicious requests.
- **Special characters are ignored.** Most existing works divide the Web request string into a word sequence (token sequence) by special characters in preprocessing, where the special characters are filtered. The filtered special characters may include important attack information [20]. Most existing works capture the feature of individual word, which ignores the relevance between words and the contextual semantic feature. It affects the detection effect of malicious Web requests.
- **Web requests are insufficient.** The number of public and private Web datasets is fewer than that of computer vision and NLP datasets in reality. Training a reasonable model with a large amount of data is neither practical nor efficient. Therefore, it is necessary to train a model with less data.

We note that Transformer can efficiently extract the long-term dependencies and complex semantic relationships of nature language. To overcome these problems, we propose an unsupervised malicious Web request detection based on transformer and contrastive learning (UTCDetector). UTCDetector exploits 2-gram, Transformer, unsupervised hypersphere loss function, and contrastive learning to detect malicious requests with no abnormal Web requests and fewer normal Web requests. The following is a summary of our major contributions:

- To extract long-term dependencies and handle complex semantic relationships, UTCDetector combines Word2vec and Transformer to effectively mine normal request patterns.
- To avoid the requirement of abnormal labels, an unsupervised hypersphere loss function is used to close the features of normal request into a hypersphere.
- To avoid filtering special characters, we propose a preprocessing method and capture the context information through 2-gram to improve the detection accuracy.
- To handle insufficient Web data, UTCDetector exploits contrastive learning to compare the feature similarity of a pair of normal requests. It is the first work on malicious web request detection.
- Since the public Web request datasets (CSIC 2010, CSIC TORPEDA 2012, and ECML/PKDD 2007) were created before 2012, we collected Web requests from a university Web application server in 2023 to build a private dataset named School 2023.[1] This dataset contains more modern and complex attacks. Our experiments on the four datasets demonstrate that UTCDetector increases the F1-score by 0.6% to 10.6% and 0.1% to 15.6% compared with the existing methods and ablation method.

The remainder of this paper is structured as follows. Section II reviews the related work. Section III defines the problem, while Section IV presents an overview and detailed steps of the unsupervised malicious Web request detection.

[1] https://github.com/AIOps-tech/School-2023-sample

Subsequently, Section V presents the experimental results and the performance analysis. The paper concludes with Section VI.

II. Related Work

We elaborate on related works in two aspects: malicious Web request detection and system log anomaly detection. System log anomaly detection shares many technologies with malicious Web request detection.

A. Malicious Web Request Detection

Supervised learning and unsupervised learning, especially natural language processing are introduced to malicious Web request detection.

Methods based on supervised learning. Liu et al. [10] propose a Web intrusion detection system combining feature analysis and SVM optimization, and establish an individual model for each type of access request. This approach belongs to machine learning. With the development of deep learning, many researchers have applied Word2vec, BERT to extract semantic feature and CNN, LSTM to detect malicious Web requests. Zhang et al. [21] use a specially designed CNN with learnable word embedding to detect Web attacks in 2017. Wan et al. [15] propose an Ensemble Learning classification system with Semantic Vectorization (ELSV), which adopts Word2Vec and TextCNN to extract semantic feature and uses XGBoost, LightGBM, and CatBoost as classification algorithms to detect normal and anomaly requests. Ma et al. [22] transform the original request into 8-dimensional request vectors, and connect multiple requests of the same user in time sequence to form user session vectors, all of which are used as the input of LSTM to detect Web attacks. Ramos Júnior et al. [16] propose LogBERT-BiLSTM to detect possible attacks on HTTP requests, which use BERT to extract semantic feature and BiLSTM to classify the logs. Kuang et al. [17] propose a model called DeepWAF, which combines LSTM and CNN to detect Web attacks. Yang et al. [23] design and apply the three-layer CNN-BiLSTM fusion attention mechanism model to the field of obfuscated malicious request detection for the first time, and consider special characters to improve accuracy. Although most above methods based on deep learning consider the semantic feature, all these methods are supervised which require abnormal label and can not identify new unknown attacks.

Methods based on unsupervised learning. SVM, random forest, clustering, and autoencoder are used in malicious Web request detection. Epp et al. [11] propose a Web firewall by specific Hypertext Transfer Protocol (HTTP) features from experts and one class SVM classifier. Cheng et al. [24] exploit pattern-tree to learn the semantic structure of URL, and analyze the value of users' input payloads (such as username, page number, delivery address, or product ID) by random forest, decision tree, SVM, or K-Nearest Neighbor (KNN). Tan and Van Hoai [9] propose an HTTP query string to numeric method, which transforms the problem of detecting "abnormal requests" into "abnormal univariate data points" and solves it by mean shift clustering algorithm in an unsupervised manner. Bronte et al. [25] propose a Web attack detection method with three feature: cross entropy of parameters, values, and data types. All the above machine learning methods depend on manual feature selection. Mac et al. [18] employ an autoencoder to detect malicious patterns in HTTP/HTTPS requests, which is a deep learning-based method. However, this approach treats URLs as character sequences based on ASCII codes. Character sequences fail to capture the crucial semantic details embedded within URLs, resulting in weak detection performance. In contrast, our method captures contextual information through 2-gram and Word2vec, enabling more accurate mining of semantic features.

B. System Log Anomaly Detection

System logs contain computer-related terms, and the detection of system logs share the same technologies with malicious Web request detection. Wibisono and Kistijantoro [26] first replace LSTM with Transformer to address the long-sequence log anomaly detection problem. Le and Zhang [27] propose a log-based anomaly detection method without log parsing (NeuralLog), which uses BERT to extract semantic feature and Transformer to detect log anomalies. Nedelkoski et al. [28] propose a classification-based method (Logsy), which combines a new hypersphere loss function with an attention-based encoder model and exploits anomaly samples from auxiliary log datasets to improve the accuracy of anomaly detection. Guo et al. [29] propose LogBERT, which acquires the log sequence vectors using the Transformer encoder and initializes the log template IDs with random vectors. Finally, it combines two self-supervised training tasks, hypersphere volume minimization and masked event prediction, to discover patterns in normal log sequences and identify anomalies. He et al. [30] propose a novel unsupervised log anomaly detection method based on multi-feature (UMFLog), which considers semantic features and statistical features to identify anomalies. He et al. [31] propose a parameter-efficient log anomaly detection scheme (LogBP-LORA) based on BERT and Low-Rank Adaptation. Inspired by log anomaly detection, we consider transform in Web request detection.

III. Preliminaries and Problem Definition

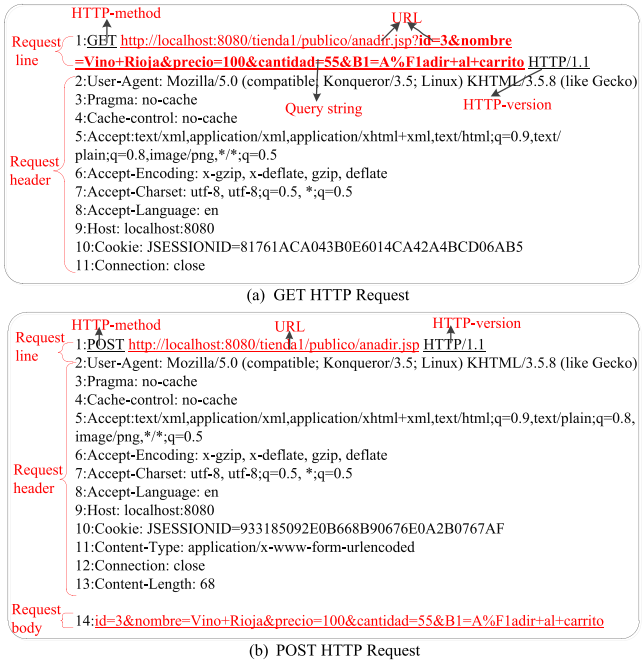
A. Web Request

Users and Web server usually use HTTP(S) protocol to exchange information and data. HTTP requests include GET, POST, PUT, DELETE, OPTIONS, HEAD, PATCH, and other methods. The GET method in HTTP is primarily used for retrieving resources (such as Web pages, images, etc.) from a server, and transmits a small amount of data through URL parameters. The POST method is mainly used for submitting data (such as form data, file uploads, etc.) to a server, and transmits a large amount of data. We take GET and POST methods as examples to illuminate the request formats. Fig. 2(a) and (b) show the GET and POST HTTP requests in the CSIC 2010 [32] datasets, respectively.

- GET request usually consists of two parts, namely, the request line and request header. The request line contains an HTTP-method 'GET', URL (scheme, host, port, path

TABLE I
COMMON SIMPLE MALICIOUS WEB REQUEST ATTACKS

Attack	Example URL	Parameter	Value
SQLi	http://www.xyz.com/login.php?uid=jonh&pwd=' or 1=1 –	uid, pwd	jonh, =' or 1=1 –
XSS	http://www.xyz.com/ index.php?uid=> <body onload= alert('test1')>&pwd=test	uid, pwd	> <body onload= alert('test1')>, test
RFI	http://www.xyz.com/test.php?uid=www.badsite.com/a.php	uid	www .badsite.com/a.php
DT	http://www .xyz.com/index. php?uid=../../../ dir/pwd.txt	uid	../../../ dir/pwd.txt



(a) GET HTTP Request

(b) POST HTTP Request

Fig. 2. The example of Web HTTP request.

and query string) and HTTP-version. The request header contains information such as Cookie and Connection.

- POST request usually consists of three parts, namely, the request line, request header, and request body. The request line of POST request is similar as that of GET request without query string. The request header is consistent with the GET method. The request body contains the data to be posting or patching.

A request body is optional because there may be nothing to express in the body of some requests, such as resource retrievals using the GET method.

Numerous malicious Web requests target different components such as URLs, request headers, and request body. Malicious Web requests can occur due to the uncontrolled user inputs that are typically included in the HTTP GET query string and POST request body. They can be exploited by attackers to create or manipulate requests to Web servers and perform malicious actions. Therefore, we focus on the URL in GET request, and the URL and request body in POST request to detect the attacks. For simplicity, we still use URL to refer to the URL and request body for POST request. Fig. 3 shows a typical URL, which consists of scheme, host, port, path, and query string. The query string includes key and value.

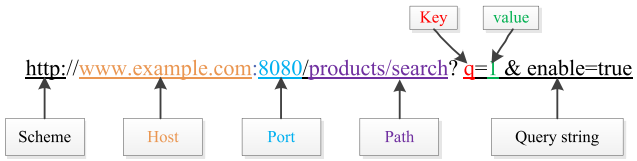


Fig. 3. The example of URL.

B. Existing Common Malicious Attacks

1) Simple attack: Many attacks primarily target the URL. Our target attacks are SQL Injection (SQLi), Cross-site Scripting (XSS), Remote File Inclusion (RFI), Directory traversal (DT), Server side include attack (SSI), XPath Injection (XPathi), Ldap Injection (Ldapi), OS Commanding, buffer overflow, information collection, file leakage, CRLF injection (CRLFi), parameter tampering, Cookie Defacement, Illegal Upload, Path Traversal, Server Information Disclosure, Web Plug-in Vulnerability, and Web Server Vulnerability attacks. We take four prevalent malicious Web request attacks, namely SQLi, XSS, RFI, and DT, as examples to illustrate the attacks, as shown in Table I.

- *SQL Injection* [33] inserts malicious SQL statements into an entry field for execution (e.g., to dump the database contents to the attacker). For example, "' or 1=1 –" lead to the request of all database, resulting in data leakage.
- *XSS attack* [34] enables attackers to inject client-side scripts into Web pages viewed by other users and bypass access controls. For example, "> < onload= alert('test1')>" is a malicious JavaScript code. In some cases, XSS attack can lead to website backdoors.
- *RFI attack* [35] enables attackers to manipulate a Web server's execution of files by constructing a path to executable code using an attacker-controlled variable. With RFI, the attacker gains the ability to determine which file is executed at runtime. For example, "www.badsite.com/a.php" contains the remote file "a.php", which introduces malicious behavior.
- *DT attack* [36] exploits weaknesses in Web applications to bypass access control mechanisms and gain unauthorized access to files or directories stored on the Web server. This type of attack aims to obtain sensitive information or perform unauthorized actions. For example, an attacker injects the directory traversal attack code "../../../ dir/pwd.txt" into the query string, which traverses to parent directory and access unauthorized "pwd.txt" file.

2) Complex attack: To bypass firewalls and Web application protection systems, there are a large number of complex malicious Web request. We analyze the characteristics of complex

TABLE II
COMMON COMPLEX MALICIOUS WEB REQUEST ATTACKS

Complex type	Raw request	Complex request
Case changes	eat	EaT
Keyword duplication	abc	ababcc
URL encoding conversion	select+1+null	select%2B1%2Bnull
Equivalent function replacement	substr('password',1)	left('password',1)
Base64 encoding conversion	username=user&password=pass	credentials=dXNlcm5hbWU9dXNlcg==&cGFzc3dvcmQ9cGFzcw==
Encryption	credit card number="1234567890123456"	encrypted_data="FE34CD90FEDCBA9876543210ABCDEF"
Combination	hex(substr((select from),1,1))>1	ascii(substr((sELeCt FroM),1,1))>1

malicious Web requests and classify them into seven types, namely case change, keyword duplication, URL encoding conversion, equivalent function replacement, Base64 encoding conversion, DES encryption, and variation. Their complex types, definitions, and examples are shown in the following and Table II.

- *Case obfuscation*. Attackers mix the case of keywords, thus bypassing some rules. For example: "eat"->"EaT".
- *Keyword duplication*. Attackers bypass filtering rules by repeatedly adding keywords. The malicious requests still work even if keywords are filtered by detection systems and attacks are launched. For example: "abc"->"ababcc".
- *URL encoding conversion*. Attackers use URL encoding to encode the requested content. For example, after converting "plus sign" into "%2B", it can avoid the detection of firewall or online WAF.
- *Equivalent function replacement*. Attackers replace some easily recognizable functions with functions that are easy to hide or functions that normal websites need to use. For example, the "substr()" function is replaced by the "left()" function.
- *Base64 encoding conversion*. Attackers use Base64 encoding to encode the requested content, which improves the compatibility of data in transmission. For example, "user" ->"dXNlcm5hbWU9dXNlcg==", "pass"->"cGFzc3dvcmQ9cGFzcw==".
- *Encryption*. Attackers use symmetric or asymmetric encryption algorithms to encrypt malicious Web requests. After receiving the requests, they decrypt and construct a payload to complete the network attack. For example, "1234567890123456"->"fe34cd90fedcba9876543210abcdef".
- *Combination*. Attackers combine several of the above complex attack methods. For example, "hex(substr((select from), 1,1)) >1" is transformed into "ascii (substr((sELeCt FroM), 1,1)) >1" by case obfuscation and function replacement.

C. Problem Definition

Suppose a datasets $D = \{S_1, S_2, \ldots, S_n\}$ contains *n* URLs. Inspired by the contrastive learning, these URLs are randomly divided into two parts D_1 and D_2 with equal data amount, where S_i represents the *ith* URL in D_1 and S_j represents the *jth* URL in D_2. The URLs of the datasets *D* are all normal. A classifier $c : S_i \rightarrow [0, 1]$ is trained to judge the newly arrived URL, where 0 is normal and 1 is abnormal.

IV. OUR UNSUPERVISED MALICIOUS WEB REQUEST DETECTION

Firstly, we give the overview of UTCDetector, and then three steps are introduced in detail.

A. Overview

To recognize unknown malicious Web requests and improve the accuracy of attack detection with fewer data and without abnormal labels, we propose unsupervised malicious Web request detection based on transformer and contrastive learning. UTCDetector utilizes preprocessing and 2-gram to retain special characters and extracts contextual semantic features of Web requests. UTCDetector leverages Transformer instead of LSTM, which extracts long-term dependencies and handles complex semantic relationships. UTCDetector exploits hypersphere loss function to replace Cross entropy loss function, which avoids the requirement of abnormal labels. UTCDetector employs contrastive learning to compare the feature similarity of paired normal requests and further narrow down the features of normal requests. This allows the UTCDetector to achieve the same good effect as a large amount of data, even with less data. UTCDetector consists of three steps: preprocessing, feature extraction, and malicious Web request detection. Fig. 4 is the framework of UTCDetector.

Preprocessing: to handle complex malicious Web requests, five procedures are used to preprocess the former five complex attacks, while the latter two complex attacks (encryption and combination) are comprehensively solved by UTCDetector.

Feature extraction: it extracts the feature representation of URL. Based on 2-gram word segmentation, special characters are reserved and the correlation between two words is established. The URL is converted into a word phrase sequence, wherein word phrase consists of two adjacent tokens, that is, special characters or words. Therefore, S_i can be represented by $[a_1^i, a_2^i, \ldots, a_{N_i}^i]$, where a_j^i is the *jth* word phrase in the *ith* URL, and N_i is the number of word phrases contained in the URL. Word2vec is used to obtain the vector representation of word phrases, that is, every word phrase is converted into a *d*-dimension vector $a_j^i \Rightarrow x_j^i$. A URL can be represented by the vector sequence of word phrases segmented by 2-gram. Since the number of words contained in a URL may be different, the word phrases generated by 2-gram may also be different. Therefore, the URL is aligned to *W* word phrases by truncating or padding to fit the following Transformer model, which is represented by $X_i = [x_1^i, x_2^i, \ldots, x_j^i, \ldots, x_W^i]$.

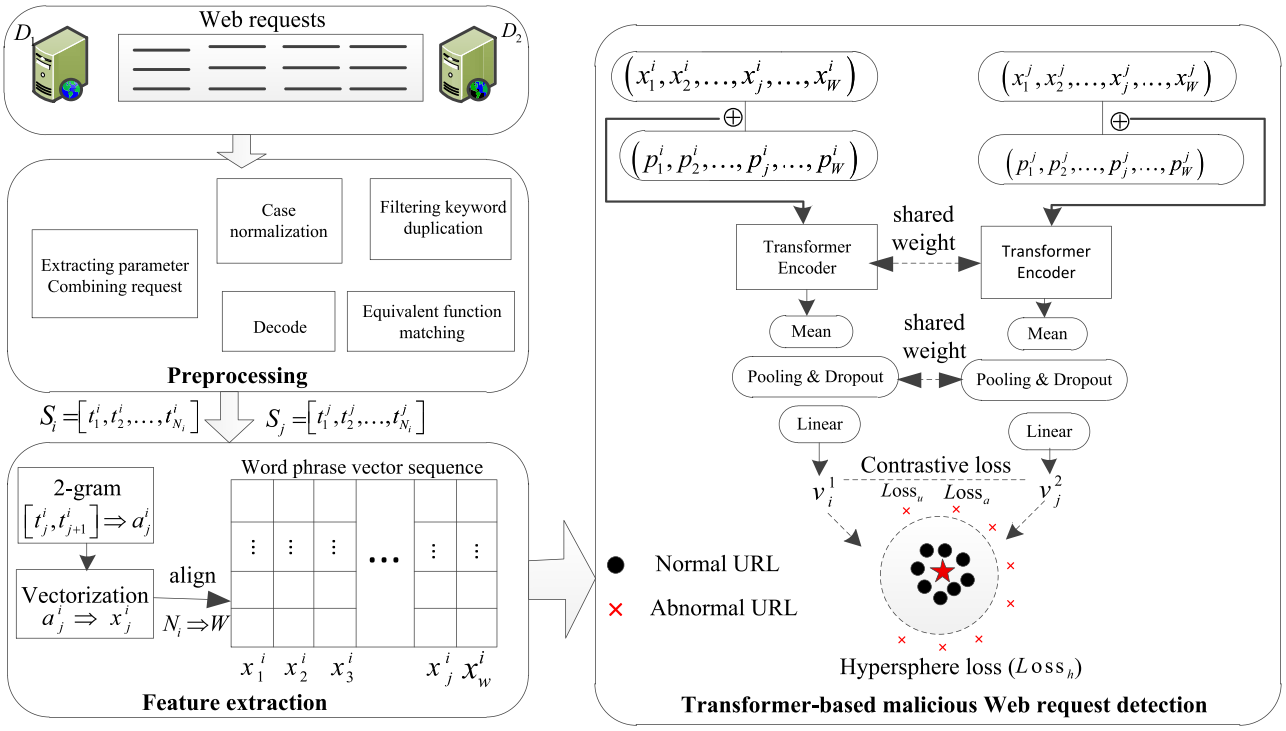


Fig. 4. The framework of UTCDetector.

Transformer-based malicious Web request detection: Transformer is trained to extract the long-term dependencies in the word phrase sequence by endogenous self-attention mechanism. Hypersphere loss function makes normal request as close to the center of the hypersphere as possible. Contrastive learning compares the feature similarity of pairs of normal requests, in which the alignment loss aligns or shortens the distance of the same pair of features, and the uniform loss makes the features evenly distributed on the hypersphere, to identify complex unknown malicious requests with less data. Then, if a newly arrived URL is located far from the hypersphere center, it is considered abnormal.

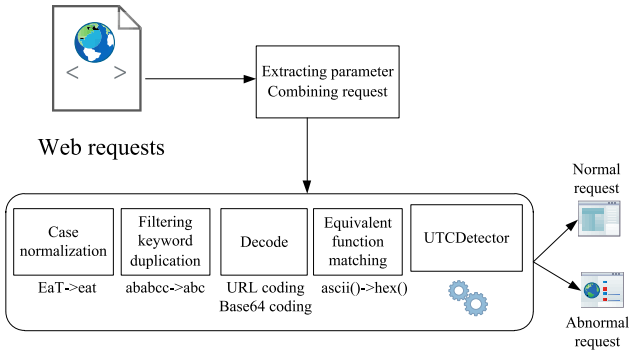


Fig. 5. Preprocessing.

B. Preprocessing

To handle complex malicious Web requests and better support the subsequent detection, we use a variety of processing inspired by [23]. The whole preprocessing flow is shown in Fig. 5, including five procedures. "encryption" and "combination" will be comprehensively solved by exploring the semantic features between contexts through UTCDetector.

1) Extracting parameters and combining requests. It extracts the URL, query string, or request body of GET and POST requests from Web requests, and then combines them into a brand-new Web request. At the same time, domain names in Web requests are excluded, because they have little influence on malicious detection of Web requests.

2) Case normalization. It makes consistent lowercase conversion for all Web requests, such as adjusting "EaT" to "eat".

3) Filtering keyword duplication. It cleans up keyword duplication in all Web requests by looping pairs, such as adjusting "ababcc" to "abc".

4) Decoding and restoring. It detects whether the Web request has undergone encoding conversion or not. If it has, the URL encoding or Base64 encoding is restored. In other words, it reverts the encoded Web request to its original form, such as "%20" as a space, "%2B" as a plus sign, and "%3D" as a "@".

5) Equivalent function matching [23]. It restores the common equivalent functions in Web malicious requests as listed in Table III, such as "ascii()" to "bin()", and "left()" to "substr()".

C. Feature Extraction

Word-level embedding may lose the semantic information of Web requests. Malicious requests usually include common special characters such as "?", "&", ".", "=", etc. When adopting special character segmentation, special characters are ignored, and only English words are preserved. Words are then

TABLE III
COMMON EQUIVALENT FUNCTION

Easily recognized function	Equivalent function easy to hide
hex()	ascii()
bin()	ascii()
mid()	substring()
substr()	substring()
substr()	left()
substr()	right()
sleep()	benchmark()
concat()	group_concat()

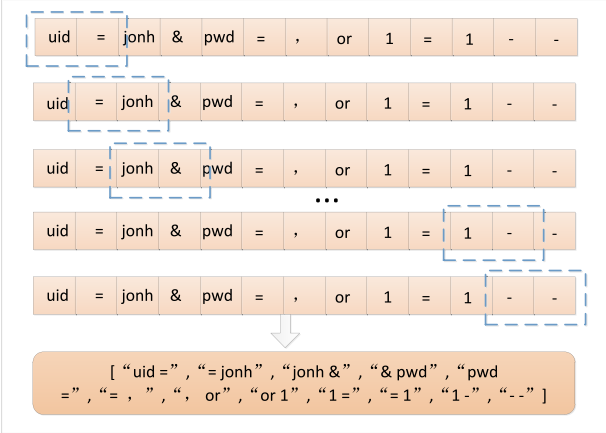


Fig. 6. 2-gram word segmentation.

converted into word vectors, but this process may result in the loss of contextual semantic relationships, which can impact the detection performance. Therefore, we exploit 2-gram and Word2vec to extract the context semantic features of Web requests.

To obtain the comprehensive characteristics of URL, we carry out word segmentation by 2-gram, preserve special characters and establish the correlation between two tokens. Word phrase consists of two special characters or words $a_j^i = [t_j^i, t_{j+1}^i]$, where t_j^i and t_{j+1}^i represent the *jth* and $(j+1)$*th* tokens of the *ith* URL, respectively. Therefore, the *ith* URL is converted into a word phrase sequence consisting of N_i word phrases, that is, $S_i = [a_1^i, a_2^i, \ldots, a_{N_i}^i]$, where a_j^i is the *jth* word phrase in the *ith* URL, $i \in [1, n]$, $j \in (1, N_i)$, and N_i is the number of word phrases in the *ith* URL. Since the number of words contained in URL may be different, the number of word phrases N_i is also different.

As shown in Fig. 6, special characters are reserved in 2-gram word segmentation. Taking the SQLi attack in Table I as an example, "or 1=1 –" is segmented into "or", "1", "=", "1", "-", "-" in 1-gram, and "or 1", "1 =", "= 1","1 -", "- -" in 2-gram, respectively. 1-gram may not be able to understand the relationship, while 2-gram can capture the connection in the context to identify the potential SQLi attack more accurately.

Then the word phrase in the word phrase sequence is converted into a corresponding vector, that is, $a_j^i \Rightarrow x_j^i$, and the word phrase sequence can be expressed as a word phrase vector sequence. Therefore, URL S_i is transformed into a word phrase vector sequence X_i, as shown in third block of Fig. 4.

Word2vec is exploited to obtain vector of word phrase after 2-gram segmentation. However, the original Word2vec vocabulary does not contain special characters. Therefore, when training Word2vec model, we extend the vocabulary to support special characters, which can effectively represent different types of complex malicious requests.

Transformer only processes unified sequences with the same length. However, the number in word phrases of URL is different. Therefore, we manually set the number of word phrases to be consistent, and truncate or fill the word phrase vector sequence of URL with an appropriate window size *W*, that is, $X_i = [x_1^i, x_2^i, \ldots, x_j^i, \ldots, x_W^i]$, where $x_j^i \in \mathbb{R}$ represent the vector of the *jth* word phrase in the *ith* URL.

D. Transformer-Based Malicious Web Request Detection

Vaswani et al. [37] introduced the Transformer model in 2017, a deep learning architecture used mainly for NLP tasks. Diverging from conventional RNNs and LSTMs, the Transformer model integrates a self-attention mechanism, enabling simultaneous processing of all sequence positions. This innovation not only bolsters training efficiency but also enables the model to effectively capture extensive dependencies within the data. With its encoder-decoder framework, the Transformer model adeptly tackles tasks such as machine translation and text generation, solidifying its position as a pivotal advancement in contemporary NLP research.

Therefore, we propose a semantic-based Transformer to detect malicious Web requests, in which Transformer is used to capture the long-distance dependencies and complex semantic information of complex malicious Web requests. For example, "union" and "select" are the characteristics of SQLi attacks. They do not appear at the same time in normal Web requests. If they appear at the same time, they can be judged as anomalies. Similarly, "< script >" and "alert" are the characteristics of XSS attacks, and they can also be judged as anomalies. Transformer reveals the strong semantic association between them. At the same time, Transformer can also find long-distance dependencies such as "dir", "../" and "txt" of DT attack in Table II, which are usually used for remote code execution attacks that bypass the path. In addition, Transformer can also deeply mine the contextual semantic features and co-occurrence features in Web requests to deal with encryption and combination attacks. As shown in Fig. 7, Transformer extracts the semantic vector sequence O_i from the word phrase vector sequence, and then the semantic vector sequence is combined into deep feature v_i to represent the URL.

The semantic vector sequence O_i is an output of the Transformer Encoder, corresponding to Eq. (1), where $O_i = [o_1^i, o_2^i, \ldots, o_j^i, \ldots, o_W^i]$. Transformer's self-attention mechanism effectively strengthens the contribution of core word phrases. The core word phrases of normal Web requests, such as "index.php", are enhanced by the attention mechanism, to better distinguish normal and malicious Web requests.

$$O_i = Encoder(X_i) \tag{1}$$

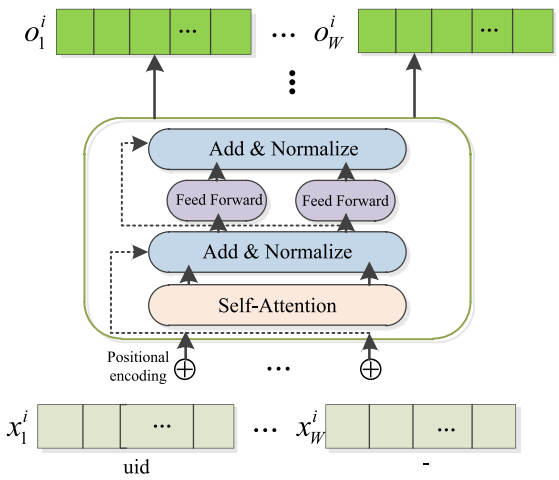


Fig. 7. Transformer Encoder.

After the semantic vector sequence is averaged, it is fed to the pooling layer, the dropout layer and the fully connected layer to obtain the deep feature v_i of the URL, as shown in Eq. (2).

$$v_i = Linear(Dropout(Pooling(Mean(O_i)))) \tag{2}$$

where *i* represents the *ith* URL.

There are a share-weight Transformer-based detection for the two datasets, as shown in four block of Fig. 4. According to the share-weight model, we can obtain the deep feature set *V1* and *V2* for datasets *D1* and *D2*, respectively. The two deep feature sets are the basis of contrastive learning.

E. Hypersphere and Contrastive Loss Function

To cluster the deep features of normal URLs, align the pairs of semantic vector sequences and retain the maximum information, the total loss function is constructed by minimizing hypersphere loss function and contrastive learning loss function (alignment loss and uniform loss). They collectively embody the most significant contribution of the entire article.

1) Hypersphere Loss Function [38]*:* Hypersphere loss is first applied in computer vision, similar to one class SVM in NLP. As shown in Fig. 4, the hypersphere loss function adjusts the distribution of all normal log sequences. Therefore, the anomaly labels are not necessary. The motivation is that the normal URLs should be concentrated in the embedded space and as close as possible to the center of the hypersphere, while the abnormal URLs should be as far away from the center of the hypersphere as possible. The best hypersphere radius (decision boundary) can be obtained by a small amount of normal and abnormal verification data. Hypersphere loss function is the mean square error sum of the distances between the samples and the center, as shown in Eq. (3).

$$\begin{aligned} Loss_s &= \sum_{v_i \in V1 \bigcup V2} \|v_i - C\|^2 \\ C &= Mean(v_i) \end{aligned} \tag{3}$$

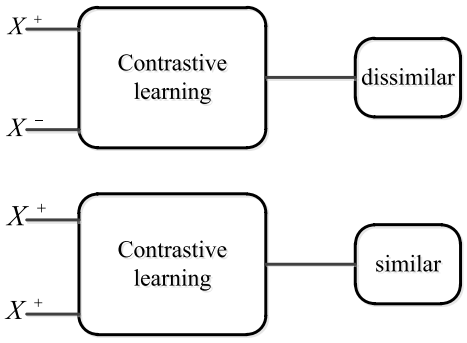


Fig. 8. The core concept of contrastive learning.

where *C* is the center of the hypersphere, *V1* and *V2* are the set of deep features, and v_i represents the deep features of the *ith* URL.

2) Contrastive Loss Function: Contrastive learning aims at learning patterns by comparing similarities or differences between two or more samples. As shown in Fig. 8, a normal sample X^+ and an abnormal sample X^- are dissimilar, while two normal samples are similar. Here, X^+ represents a normal sample, and X^- represents an abnormal sample. Contrastive learning is usually used in target detection, image classification, recommendation system and other fields. The advantage of contrastive learning is to maximize the distance between positive and negative samples while minimizing the intra-class distance within positive or negative samples, where positive samples represent normal Web requests and negative samples represent abnormal Web requests. Therefore, samples of the same kind can be close, and samples of different kinds can be far away. In order to assign similar features to samples of the same kind and keep the feature distribution as much as possible, we introduce alignment loss and uniformity loss. Alignment loss aligns or shortens the feature distance of similar sample pairs, and uniformity loss makes the features evenly distributed on the hypersphere.

The alignment loss function is shown in Eq. (4).

$$Loss_a = \sum_{i=1}^{n/2} \sum_{j=1}^{n/2} \left[\|v_i^1 - v_j^2\|_2^2 \right] \tag{4}$$

where v_i^1 represents the deep feature of the URL in D_1, v_j^2 represents the deep feature of the URL in D_2, $n/2$ represents the number of URLs in D_1 and D_2, and $\|\cdot\|_2$ represents the Frobenius norm, respectively.

We want the uniformity metric to be both asymptotically valid (i.e., the distribution that optimizes this metric should converge to uniform distribution) and empirically plausible with small number of points. To this end, we consider the Gaussian potential kernel (also known as the Radial Basis Function kernel) and define the uniformity loss as the logarithm of the average pairwise Gaussian potential [39]. The uniformity loss function is shown in Eq. (5).

$$Loss_u = log \sum_{i=1}^{n/2} \sum_{j=1}^{n/2} \left[e^{-2\|v_i^1 - v_j^2\|_2^2} \right] \tag{5}$$

TABLE IV
WEB DATASETS

Dataset	Size	#All Requests	#Anomalies	#Training	#Testing (Nor/Abnor)
CSIC 2010 [32]	97K	97000	25000	36000	36000/25000
CSIC TORPEDA 2012 [40]	100K	74133	65770	4182	4181/65770
ECML/PKDD 2007 [41]	50K	50116	15110	24504	10502/15110
School 2023	23757K	124746	25000	49880	49866/25000

The definition of the total loss function is shown in Eq. (6).

$$Loss_{sum} = Loss_s + \alpha_1 * Loss_a + \alpha_2 * Loss_u \quad (6)$$

where α_1 and α_2 are hyperparameters for balancing the three loss functions.

According to the above steps, we can train UTCDetector. When a new URL arrives, it is also preprocessed and extracted feature into a word phrase vector sequence, and then input into the trained Transformer model to obtain the deep feature. UTCDetector can predict whether the URL is abnormal according to the distance between the deep feature and the hypersphere center.

V. PERFORMANCE EVALUATION

In this section, we evaluate our model by answering the following research questions (RQs):

RQ1: How does our method perform in malicious Web request detection?

RQ2: How about the ablation experiment?

RQ3: How sensitive is our method?

RQ4: How about attack case studies?

A. Experimental Setup

1) Datasets: We conducted our experiments primarily on three public datasets, namely CSIC 2010, CSIC TORPEDA 2012, and ECML/PKDD 2007. Since the three public datasets are before 2012, we collected the Web requests from a university Web application server in 2023, called School 2023. The statistical information of four datasets is as shown in Table IV.

CSIC 2010 [32]. It was developed by the Institute of Information Security of the Spanish National Research Council (CSIC), which contains thousands of automatically created Web requests. It can be utilized to test the Web attack protection systems. Because it is a Spanish Web application, the datasets contains some Latin characters. This dataset contains the traffic generated for e-commerce Web applications, in which users can use shopping carts to buy goods and register by providing some personal information. This dataset contains 72,000 normal requests and more than 25,000 abnormal requests (including SQL Injection, buffer overflow, information collection, file leakage, CRLFi, XSS, server-side inclusion, parameter tampering, and other attacks).

CSIC Torpeda 2012 [40]. It is constructed by TORPEDA rules and artificially generated with the help of semi-automatic tools. All requests point to an e-commerce Web application, which is developed for testing purposes and has obvious loopholes. The dataset consists of 8363 normal requests, 16459 abnormal requests, and 49311 malicious requests (43013 SQLi and variants, 4818 XSS and variants, 412 buffer overflows, 41 format strings, 74 LDAPi, 451 SSI, 175 XPath and 327 CRLFi).

ECML/PKDD 2007 [41]. It was put forward by the 18th European Machine Learning Conference (ECML) and the 11th European Conference on Knowledge Discovery in Database (PKDD) in 2007. It is created by capturing actual traffic and then processed to purify information, including requests classified into seven different types of attacks. The datasets include 35,006 requests categorized as normal and 15,110 requests classed as attacks.

School 2023. We collected the access Web request from a university Web application server in June 2023, which contains 99746 normal requests and 25000 malicious requests (including ten types attacks: SQLi, XSS, RFI, OS Commanding, Cookie Defacement, Illegal Upload, Path Traversal, Server Information Disclosure, Web Plug-in Vulnerability, and Web Server Vulnerability attacks). We divide these Web requests into three files, namely, normal training, normal testing and abnormal testing files.

2) Baselines: Our method is compared with other existing methods One-Class SVM [11], Pattern-tree [24], HQTN [9], Autoencoder [18], OMRDetector [23].

- **One-Class SVM** [11] uses the expert knowledge of HTTP requests to manually build features and then uses a One-class SVM classifier to detect abnormal requests.
- *Pattern-tree* [24] manually extracts key-value pair information and matches it through a pattern tree, and then uses models such as random forest, decision tree, or support vector machine to detect anomalies.
- *HQTN* [9] converts the string of attribute names and attribute values into numbers and then uses the mean shift clustering algorithm to detect anomalies in an unsupervised way.
- **Autoencoder** [18] tokenizes the URL and then uses autoencoder to calculate the reconstruction error of a given request, find the threshold, and judge whether it is abnormal or not.
- **OMRDetector** [23] applies the three-layer CNN-BiLSTM to obtain the local characteristics and contextual semantic features of Web requests, and finally the detection results of malicious requests are calculated by Softmax classifier.

As the existing methods are not open-source, we refer to their experimental results directly in our work and report their best results.

3) Implementation and Environment: All experiments are run on the Google Colab cloud platform, which provides an online deep learning server supported by Intel Xeon Gold 6148 CPU, NVIDIA A100-SXM, and 85GB RAM with eight cores. The word embedding dimension Emb_dim in Word2vec is 300, and Transformer is a basic model, in which the dimension of Transformer's hidden layer H = 128, the number of multi-head self-attention heads N=12, and the default experiment parameters are set as follows: max_epoch=100, $batch\_size$=64. The window size W is set

TABLE V
RESULTS OF COMPARISON WITH EXISTING METHODS ON PUBLIC DATASETS

Dataset	Method	Precision	Recall	F1-score
CSIC 2010	One-class SVM	\	\	0.93 ± 0.07
	Pattern-tree	0.93	0.85	0.89
	HQTN	\	\	0.99
	Autoencoder	0.9464	0.9462	0.9463
	OMRDetector (supervised)	0.97734	0.97737	0.97735
	UTCDetector	**0.99366**	**1**	**0.99682**
CSIC TORPEDA 2012	One-class SVM	\	\	0.93 ± 0.07
	UTCDetector	**0.9812**	**1**	**0.99051**
ECML/PKDD 2007	LogBERT-BiLSTM (supervised)	0.96	0.97	0.97
	UTCDetector	**0.9785**	**1**	**0.98913**

to cover 90% URLs, which are 40, 20, 40 and 20 for four datasets, respectively.

4) Metrics: This paper uses Precision, Recall, F1-score, and training time for performance analysis. Precision represents the ratio of how many detected anomalies are true, and Recall represents the ratio of how many reported anomalies are true. Finally, the F1-score is the harmonic average of the accuracy and recall in the equation, as shown in Eq. (7), where the best value is 1 and the worst value is 0. Training time refers to the time required to train the model.

$$\begin{aligned} Precision &= \frac{True\ Positive}{True\ Positive + False\ Positive} \\ Recall &= \frac{True\ Positive}{True\ Positive + False\ Negetive} \\ F1{-}score &= \frac{2 * precision * recall}{precision + recall} \end{aligned} \tag{7}$$

B. RQ1: How Does Our Method Perform in Malicious Web Request Detection?

The performance comparison with the existing methods on three public datasets is shown in Table V, where \ means that the original works do not provide the result of this metric. For CSIC 2010 and CSIC TORPEDA 2012 datasets, One-class SVM, Pattern-tree, and HQTN achieve low detection performance (0.89-0.99). They are machine learning methods that depend on manual feature extraction. The F1-score of Autoencoder is only 0.9463. The reason is that it has poor generalization ability when the testing samples and training samples have different distributions. For the ECML/PKDD 2007 dataset, we compare UTCDetector with a supervised method (LogBERT-BiLSTM). LogBERT-BiLSTM [16] uses BERT to extract semantic feature and BiLSTM to classify the logs with labels. As shown in Table V, our method can achieve a higher F1-score (0.98931) than LogBERT-BiLSTM (0.97) without anomaly label, which greatly reduces the consumption of large amounts of labor and time.

C. RQ2: How About the Ablation Experiment?

We conduct two ablation experiments from the aspects of feature extraction, detection, and contrastive learning.

Firstly, our method is compared with three methods with different feature extraction models and detection models: Word2vec-LSTM, BERT-Transformer, and BERT-LSTM. Transformer is trained from scratch which is same as our method, while BERT is a pre-trained model.

- *Word2vec-LSTM* converts words into vectors by word2vec and then uses the LSTM model to detect malicious Web request.
- *BERT-Transformer* converts words into vectors by an off-the-shelf service BERT-as-service, and then exploits the Transformer model to detect malicious Web request. Although BERT can be fine-tuned, BERT is utilized directly without fine-tuning.
- *BERT-LSTM* converts words into vectors by an off-the-shelf service BERT-as-service, and then exploits the LSTM model to detect malicious Web request.

The performance on four datasets is shown in Table VI, in which all the results are taken from the average after five experiments. The performance of Word2vec-LSTM, BERT-Transformer, and BERT-LSTM is unstable. In the CSIC 2010 dataset, the F1-scores of BERT-Transformer and BERT-LSTM are over 0.95. In the CSIC TORPEDA 2012 dataset, the F1-scores of Word2vec-LSTM and BERT-Transformer are over 0.95. However, in the ECML/PKDD 2007 dataset, the F1-scores of the three methods are under 0.78. In School 2023 dataset, the F1-scores of BERT-Transformer and BERT-LSTM are over 0.99. The reason is that LSTM cannot effectively extract long-term dependencies of long sequence data, and the off-the-shelf service BERT-as-service is pre-trained by a general corpus which is not suitable for URLs.

UTCDetector achieves the best F1-scores on the CSIC TORPEDA 2012, ECML/PKDD 2007 datasets, and School 2023. UTCDetector exhibits a slight decrease compared to the BERT-Transformer on the CSIC 2010 dataset, it is almost negligible. The variance results prove that its robust performance across various datasets and attack scenarios is excellent. The main reasons are that the Word2vec in UTCDetector is suitable for both small and large datasets, and Transformer can take into account the semantic and relative position information of words.

To demonstrate the efficiency of UTCDetector, we compare the testing time of the four methods on the CSIC 2010 dataset. As shown in Table VII, it is evident that our method is the fastest at detecting malicious web requests.

Secondly, to analyze the contributions of contrastive learning, we compare the method with and without contrastive learning. UTCDetector w/o CL represents UTCDetector without contrastive learning loss function. The experimental results on four datasets are shown in Table VIII, where training data represents the number of training URLs. When the similar F1-score are achieved, the required training data of UTCDetector is nearly half of that without contrastive learning. The reason is that contrastive learning compares the feature similarity of a

TABLE VI
ABLATION EXPERIMENT ON ALL DATASETS

Method	CSIC 2010			CSIC TORPEDA 2012			ECML/PKDD 2007			School 2023		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Word2vec-LSTM	0.77994	0.9092	0.8396	0.9885	0.98382	0.98615	0.97892	0.65473	0.78466	0.96586	0.96177	0.96381
BERT-Transformer	1	0.99784	**0.99892**	0.97864	0.9337	0.95564	0.71102	0.7312	0.72097	0.99496	0.99134	0.99315
BERT-LSTM	0.96757	0.94801	0.95769	0.99681	0.76191	0.86367	0.65364	0.32701	0.43593	0.99383	0.98829	0.99105
UTCDetector	0.99366	1	0.99682±0.002	0.9812	1	**0.99051±0.001**	0.9785	1	**0.98913±0.003**	0.99752	0.99343	**0.99547±0.002**

TABLE VII
TESTING TIME OF ABLATION METHODS ON CSIC 2010 DATASET

Method	Testing time(s)
Word2vec-LSTM	22.93
BERT-Transformer	13.2
BERT-LSTM	31.29
UTCDetector	10.11

TABLE VIII
THE IMPACT OF CONTRASTIVE LEARNING

Dataset	CSIC 2010		CSIC Torpeda 2012		ECML/PKDD 2007		School 2023	
Training data	20000	36000	5000	8363	14000	24504	29880	49880
UTCDetector w/o CL	0.98376	0.99682	0.96907	0.99051	0.89	0.98913	0.98897	0.99346
UTCDetector	**0.99682**	0.99682	**0.99051**	0.991	**0.98913**	0.98982	**0.99547**	0.99798

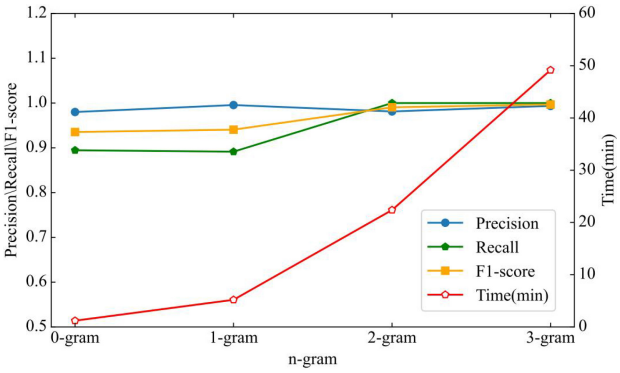


Fig. 9. Impact of n on CSIC TORPEDA 2012 dataset.

pair of normal requests and further close the feature of normal requests [39].

D. RQ3: How Sensitive Is Our Method?

Four parameters are considered to analyze the robustness of UTCDetector: window size, Word2vec embedding dimension, The dimension of the Transformer's hidden layer, and the number of Transformer heads.

Impact of different n-gram. To verify the influence of the size of n in n-gram on the experimental results, we set n to 0 to 3, where 0 means no special characters. As shown in Fig. 9, with the increase of n, the F1-score also gradually increases. The greater the n, the more the number of adjacent words or characters in the text, the closer the context and the clearer the semantics. Therefore, the detection effect is better. However,

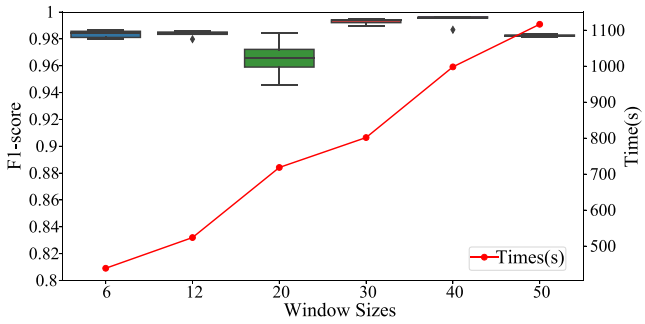


Fig. 10. Impact of window size on CSIC 2010 datasets.

the larger n is, the more word phrases are included in the vocabulary, which leads to an increase in the time required to train the Word2vec model. Therefore, it is not necessarily the case that a larger value of n yields superior performance. To sum up, we choose the 2-gram word segmentation method which can just balance the time and F1-score.

Impact of window size. The window size W is set from 6 to 50. As shown in Fig. 10, with the change in window size, the differences in the F1-score are not large. The best F1-score (0.99682) is with a length of 40, which is longer than 90% URL. If the window size is too small, attack information may be lost due to truncation which leads to low performance. If the window is too large, the word phrase vector sequence is filled with lots of zero and becomes sparse, which affects the detection performance. At the same time, the training time increases linearly with the increase in the window size. The more word phrases are included in the window size, the longer time it takes to train the model. However, the larger the window size does not mean the higher the performance according to Fig. 10. Therefore, we set the window size to 40 which can obtain the best performance and take a relatively short time.

Impact of Word2vec embedding dimension. The Word2vec embedding dimension emb_dim is set from 24 to 960. As shown in Fig. 11, the Precision, Recall and, F1-score increase with the increase of the embedding dimension of Word2ve. It reaches the peak at 300 (0.99682) and maintains the level. Word2vec with more than 120 dimensions can capture sufficient semantic information and features, thereby enhancing the model's accuracy in distinguishing between normal and abnormal requests.

Impact of the dimension of the Transformer's hidden layer. The dimension of the Transformer's hidden layer $Hidden\_layer$ is set from 32 to 2048. As shown in Fig. 12, the

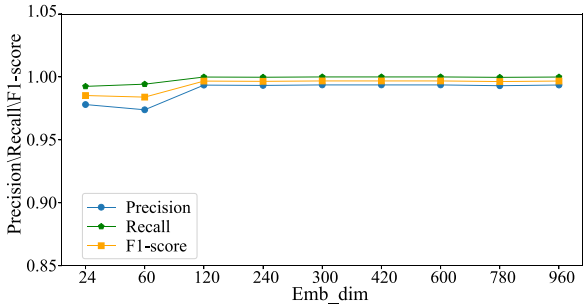


Fig. 11. Impact of embedded dimension on CSIC 2010 datasets.

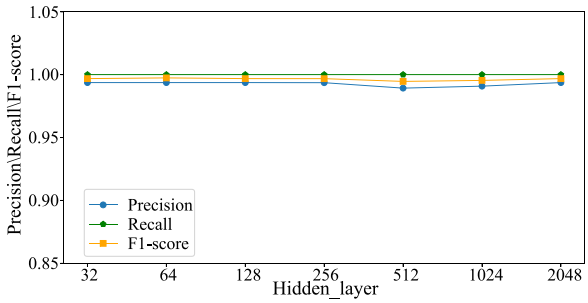


Fig. 12. Impact of the dimension of hidden layer on CSIC 2010 dataset.

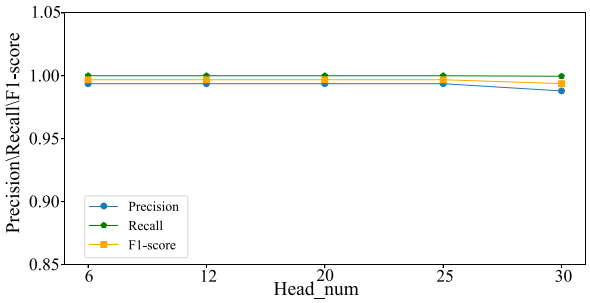


Fig. 13. Impact of the number of heads on CSIC 2010 dataset.

Precision, Recall, and F1-score remain basically stable with the increase of the dimension of the Transformer's hidden layer and reach the peak at 64 (0.99682).

Impact of the number of Transformer heads. The dimension of the Transformer's hidden layer *Head_num* is set from 6 to 30. As shown in Fig. 13, the Precision, Recall, and F1-score remain basically stable (0.99682) with the increase in the number of Transformer heads. The multi-head attention mechanism excels at comprehensively capturing feature information even with a limited number of heads. The number of heads has little effect on the performance improvement.

E. RQ4: How About Attack Case Studies?

We use the detection ratio to evaluate the detection performance of different attacks on ECML/PKDD 2007. The detection ratio is equal to the number of correctly identifying such attacks as anomalies divided by the total number of such attacks. ECML/PKDD 2007 includes seven kinds of attacks: SQLi, XSS, DT, SSI, XPathi, Ldapi, and OS Commanding, where the latter four attacks are also mainly involved in the URL. All kinds of attacks can be detected by our method and the detection ratio is 1, that is, all kinds of attacks can be detected correctly.

TABLE IX
ATTACK CASE STUDIES ON ECML/PKDD 2007 DATASET

Attacks	Detected?	Ratio
SQLi	Yes	1
XSS	Yes	1
DT	Yes	1
SSI	Yes	1
XPathi	Yes	1
Ldapi	Yes	1
OS Commanding	Yes	1

VI. CONCLUSION

Malicious Web request detection is very important to protect computer systems from malicious attacks. This paper proposes the unsupervised malicious Web request detection based on transformer and contrastive learning. It exploits preprocessing and 2-gram to preserve special characters and uses Transformer to extract complex semantic relationships. Hypersphere and contrastive loss function are introduced to identify unknown attacks instead of cross entropy loss function, which facilitates unsupervised detection with less training data. The experimental results on four Web datasets show that our method is superior to the existing methods in detecting malicious Web requests. In our future work, we aim to integrate machine learning techniques with in-depth cybersecurity expertise. We plan to evaluate our approach on a more diverse range of datasets, with a particular focus on real-world modern Web traffic dynamics. Additionally, we intend to explore the integration of our method into existing network security systems.

REFERENCES

[1] CNCERT. "2022 interconnection security report." 2022. [Online]. Available: https://www.fisec.cn/
[2] L. Yu et al., "Detecting malicious Web requests using an enhanced textCNN," in *Proc. IEEE 44th Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, 2020, pp. 768–777.
[3] S. Axelsson, "Research in intrusion-detection systems: A survey," Dept. Comput. Eng., Chalmers Univ. Technol., Gothenburg, Sweden, Rep. 98–17, 1998.
[4] P. Garcia-Teodoro, J. Diaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," *Comput. Secur.*, vol. 28, nos. 1–2, pp. 18–28, 2009.
[5] S. He, G. Li, K. Xie, and P. K. Sharma, "Fusion graph structure learning-based multivariate time series anomaly detection with structured prior knowledge," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8760–8772, 2024.
[6] S. He, Q. Guo, G. Li, K. Xie, and P. K. Sharma, "Multivariate time series anomaly detection based on multiple spatio-temporal graph convolution," *IEEE Trans. Instrum. Meas.*, vol. 74, pp. 1–14, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10766359
[7] S. He et al., "Graph structure learning-based multivariate time series anomaly detection in Internet of Things for human-centric consumer applications," *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 5419–5431, Aug. 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10547539
[8] J. Liu et al., "A generic framework for finding special quadratic elements in data streams," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 3269–3284, Aug. 2024.

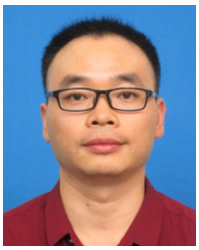
[9] H. H. Tan and T. Van Hoai, "Web application anomaly detection based on converting HTTP request parameters to numeric," in *Proc. 15th Int. Conf. Adv. Comput. Appl. (ACOMP)*, 2021, pp. 93–97.
[10] C. Liu, J. Yang, and J. Wu, "Web intrusion detection system combined with feature analysis and SVM optimization," *EURASIP J. Wireless Commun. Netw.*, vol. 2020, no. 1, Feb. 2020, Art. no. 5750646.
[11] N. Epp, R. Funk, C. Cappo, and S. Lorenzo-Paraguay, "Anomaly-based Web application firewall using http-specific features and one-class SVM," in *Proc. Workshop Regional de Segurança da Informação e de Sistemas Computacionais*, 2017, pp. 1–11.
[12] J. Liang, W. Zhao, and W. Ye, "Anomaly-based Web attack detection: A deep learning approach," in *Proc. VI Int. Conf. Netw., Commun. Comput.*, 2017, pp. 80–85.
[13] M. Zhang, S. Lu, and B. Xu, "An anomaly detection method based on multi-models to detect Web attacks," in *Proc. 10th Int. Symp. Comput. Intell. Design (ISCID)*, 2017, pp. 404–409.
[14] R. Tang et al., "Zerowall: Detecting zero-day Web attacks through encoder-decoder recurrent neural networks," in *Proc. INFOCOM IEEE Conf. Comput. Commun.*, 2020, pp. 2479–2488.
[15] W. Wan, X. Shi, J. Wei, J. Zhao, and C. Long, "ELSV: An effective anomaly detection system from Web access logs," in *Proc. IEEE Int. Perform., Comput., Commun. Conf. (IPCCC)*, 2021, pp. 1–6.
[16] L. S. Ramos Júnior, D. Macêdo, A. L. Oliveira, and C. Zanchettin, "LogBERT-BiLSTM: Detecting malicious Web requests," in *Proc. 31st Int. Conf. Artif. Neural Netw. (ICANN)*, Bristol, U.K., 2022, pp. 704–715.
[17] X. Kuang et al., "DeepWAF: Detecting Web attacks based on CNN and LSTM models," in *Proc. 11th Int. Symp. Cyberspace Saf. Secur. (CSS)*, Guangzhou, China, 2019, pp. 121–136.
[18] H. Mac, D. Truong, L. Nguyen, H. Nguyen, H. A. Tran, and D. Tran, "Detecting attacks on Web applications using autoencoder," in *Proc. 9th Int. Symp. Inf. Commun. Technol.*, 2018, pp. 416–421.
[19] S. He et al., "A joint matrix factorization and clustering scheme for irregular time series data," *Inf. Sci.*, vol. 644, Oct. 2023, Art. no. 119220.
[20] T. Ji et al., "Research on deep learning-powered malware attack and defense techniques," *Chin. J. Comput.*, vol. 44, no. 4, pp. 669–695, 2021.
[21] M. Zhang, B. Xu, S. Bai, S. Lu, and Z. Lin, "A deep learning method to detect Web attacks using a specially designed CNN," in *Proc. 24th Int. Conf. Neural Inf. Process. (ICONIP)*, Guangzhou, China, 2017, pp. 828–836.
[22] H. Ma, C. Wang, and H. Qi, "Anomaly Behavior detection for the Web application based on LSTM," in *Proc. IEEE Conf. Telecommun., Opt. Comput. Sci. (TOCS)*, 2021, pp. 553–559.
[23] X. Yang, G. Peng, Y. Luo, W. Song, J. Zhang, and F. Cao, "OMRDetector: A method for detecting obfuscated malicious requests based on deep learning," *Chin. J. Comput.*, vol. 45, no. 10, pp. 2167–2189, 2022.
[24] Z. Cheng, B. Cui, T. Qi, W. Yang, and J. Fu, "An improved feature extraction approach for Web anomaly detection based on semantic structure," *Secur. Commun. Netw.*, vol. 2021, no. 1, Feb. 2021, Art. no. 6661124.
[25] R. Bronte, H. Shahriar, and H. Haddad, "Information theoretic anomaly detection framework for Web application," in *Proc. IEEE 40th Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, 2016, pp. 394–399.
[26] S. R. Wibisono and A. I. Kistijantoro, "Log anomaly detection using adaptive universal transformer," in *Proc. Int. Conf. Adv. Inform., Concepts, Theory Appl. (ICAICTA)*, 2019, pp. 1–6.
[27] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*, 2021, pp. 492–504.
[28] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Self-attentive classification-based anomaly detection in unstructured logs," in *Proc. IEEE Int. Conf. Data Min. (ICDM)*, 2020, pp. 1196–1201.
[29] H. Guo, S. Yuan, and X. Wu, "Logbert: Log anomaly detection via bert," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2021, pp. 1–8.
[30] S. He, D. Tuo, B. Chen, R. S. Sherratt, and J. Wang, "Unsupervised log anomaly detection method based on multi-feature," *Comput., Mater. Continua*, vol. 99, no. 1, pp. 1–20, 2023.
[31] S. He, Y. Lei, Y. Zhang, K. Xie, and P. K. Sharma, "Parameter-efficient log anomaly detection based on pre-training model and LORA," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*, 2023, pp. 207–217.
[32] C. T. Giménez, A. P. Villegas, and G. Á. Marañón, *HTTP Data Set CSIC 2010 Information Security Institute CSIC*, vol. 64, Spanish Res. Nat. Council, Madrid, Spain, 2010.
[33] OWASP. "SQL injection." 2021. [Online]. Available: https://owasp.org/www-community/attacks/SQL_Injection
[34] A. Hannousse, S. Yahiouche, and M. C. Nait-Hamoud, "Twenty-two years since revealing cross-site scripting attacks: A systematic mapping and a comprehensive survey," *Comput. Sci. Rev.*, vol. 52, no. 1, pp. 1–52, 2024.
[35] OWASP. "Server-side includes (SSI) injection." 2016. [Online]. Available: https://owasp.org/www-community/attacks/Server-Side_Includes_(SSI)_Injection
[36] OWASP. "Path traversal." 2016. [Online]. Available: https://owasp.org/www-community/attacks/Path_Traversal
[37] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
[38] X. Han and S. Yuan, "Unsupervised cross-system log anomaly detection via domain adaptation," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 3068–3072.
[39] T. Wang and P. Isola, "Understanding contrastive representation learning through alignment and uniformity on the hypersphere," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 9929–9939.
[40] (ITI-CERT, Basel, Switzerland, Tampere Univ., Tampere, Finland). *Torpedo-Automated Framework for Exploit Generation*. 2017. [Online]. Available: http://www.tic.itefi.csic.es/torpeda
[41] C. Raıssi, J. Brissaud, G. Dray, P. Poncelet, M. Roche, and M. Teisseire, "Web analyzing traffic challenge: Description and results," in *Proc. ECML/PKDD*, 2007, pp. 47–52.

Shiming He received the B.S. degree in information security and the Ph.D. degree in computer science and technology from Hunan University, China, in 2006 and 2013, respectively.

She is currently a Professor with the School of Computer Science and Technology, Changsha University of Science and Technology, Changsha, China. Her research interests include machine learning, data analysis, and anomaly detection.

Ying Zhang received the M.S. degree in computer technology from the Changsha University of Science and Technology, China, in 2024.

She is currently working with the Institute of Information Engineering, Hunan University of Science and Engineering, Yongzhou, China. Her research interests mainly include deep learning, data analysis, and anomaly detection.



Diqing Liang received the Ph.D. degree in computer software and theory from Central South University, China, in 2017.

He is currently a Senior Engineer with the Information Construction Management Department, Changsha University of Science and Technology, Changsha, China. Research interests include artificial intelligence, big data, and cyberspace security.

Pradip Kumar Sharma (Senior Member, IEEE) received the Ph.D. degree in CSE from the Seoul National University of Science and Technology, South Korea, in August 2019.

He is an Assistant Professor of Cybersecurity with the Department of Computing Science, University of Aberdeen, U.K. He also worked as a Postdoctoral Research Fellow with the Department of Multimedia Engineering, Dongguk University, South Korea. He was a Software Engineer with MAQ Software, India, and involved on variety of projects, proficient in building largescale complex data warehouses, OLAP models, and reporting solutions that meet business objectives and align IT with business. He has published many technical research papers in leading journals from IEEE, Elsevier, Springer, and MDPI. Some of his research findings are published in the most cited journals. His current research interests are focused on the areas of cybersecurity, blockchain, edge computing, SDN, and IoT security. He is listed in the world's Top 2% Scientists for citation impact during the calendar year 2019 by Stanford University. Also, he received a top 1% reviewer in computer science by Publons Peer Review Awards 2018 and 2019, Clarivate Analytics. He has been an expert reviewer for IEEE Transactions, Elsevier, Springer, and MDPI journals and magazines. He has also been invited to serve as a Technical Programme Committee Member and the Chair in several reputed international conferences, such as IEEE DASC 2021, IEEE CNCC 2021, CSA 20202, IEEE ICC2019, IEEE MENACOMM'19, and 3ICT 2019. He is currently an Associate Editor of *Peer-to-Peer Networking and Applications*, *Human-centric Computing and Information Sciences*, *Electronics* (MDPI), and *Journal of Information Processing Systems* (JIPS). He has been serving as a Guest Editor for international journals of certain publishers, such as IEEE, Elsevier, Springer, MDPI, and JIPS.